

Decoding Binary Linear Block Codes Using Local Search

M. Esmaeili, *Member, IEEE*, A. Alampour, and T. Aaron Gulliver, *Senior Member, IEEE*

Abstract—This paper presents a novel iterative hard decision decoding algorithm for binary linear block codes over a binary symmetric channel (BSC). The problem is formulated as a 0-1 integer programming problem which is known to be NP-hard. When the crossover probability ϵ of the channel is known, the solution space of the decoding problem can be decreased to a sphere whose radius is related to ϵ . Using the penalty function method, the problem is reformulated on this reduced solution space. Then an iterative multi-flip local search algorithm is designed to find the global solution of this decoding problem. For a code with minimum distance d , when the radius of the sphere is not greater than $\lfloor \frac{d-1}{2} \rfloor$, this algorithm has the maximum likelihood (ML) certificate property, i.e., if the decoder outputs a codeword, it is guaranteed to be the ML codeword. Compared to the probabilistic suboptimal iterative belief propagation (BP) decoder, this approach has lower complexity and better performance. Numerical results show that in terms of speed and performance the proposed decoding method outperforms BP decoding in the error floor region.

Index Terms—Integer programming decoding, maximum likelihood hard decision decoding, iterative decoding, multi-flip local search decoding.

I. INTRODUCTION

DIGITAL wireless communications involves the transmission of data over noisy channels. Error correcting codes are employed to correct the resulting errors, thus information is sent over a communications channel using codewords. A decoder is an algorithm which tries to determine the transmitted codeword from the received noisy word. In general, decoders can be classified into two main groups, hard decision input decoders and soft decision input decoders. In this paper, we focus on hard decision decoding and present a new nonlinear integer programming model.

For linear block codes, the maximum likelihood decoding (MLD) problem can be formulated as an integer programming (IP) problem which belongs to the class of NP-hard problems [1]. Almost all models of the MLD problem are formulated based on integer linear programming (ILP). A numerical comparison of IP formulations and a survey of mathematical

programming decoding of binary linear codes can be found in [6] and [7]. In 1998, Breithach et al. were the first to model the MLD problem as an ILP problem for soft decision decoding [2]. In 2005, three alternative ILP formulations were proposed by Feldman et al. to solve the MLD problem [3]. In 2008, Yang et al. [4] presented a new ILP formulation by decomposing a high degree check node into several low degree check nodes such that the degree of each check node is less than 3. In 2010, Tanatmis et al. [5] considered the ILP formulations in [2] and devised a method to separate the original problem into subproblems to improve decoding performance. The solution of these ILP formulations was based on LP relaxation. In [6], Tanatmis et al. compared the complexity of these ILP models and ML decoding with respect to the code length and the number of parity check equations using the general purpose software CPLEX. In general, the main disadvantage of LP based decoders is the increase in the number of variables or constraints due to modeling. Moreover, LP based decoders do not always produce integral codewords.

For a binary code of length n and dimension k , as n and k increase the major difficulties with these models are:

1. an increase in the number of codewords with 2^k points as part of the local minimizers in the solution space;
2. a larger number of constraints or variables; and a large solution space size.

Another class of decoders is the iterative decoders, which includes belief propagation based (BP) algorithms [8] and bit flipping (BF) algorithms (e.g. [9] and [10]). Although BP decoding algorithms achieve near capacity performance, their complexity is very high even considering implementation in hardware [11]. Conversely, BF algorithms can be efficiently implemented and have significantly lower decoding complexity compared with BP decoding. However, BF algorithms have rather poor error correcting performance (e.g. [9], [10], and [12]).

The goal in this paper is to address the MLD problem from a combinatorial optimization viewpoint. We develop an algorithm which is similar to BF algorithms in that it can be implemented efficiently in hardware, and also has performance better than a BP decoder.

We limit the size of the solution space for a code according to the channel crossover probability, and contract the MLD problem to a restricted space. Then a new IP formulation is proposed using the penalty function method, and an iterative multi-flip local search algorithm is designed. Compared to the standard BP decoder, our multi-flip local search decoder has better performance in the error floor region with lower complexity.

The remainder of this paper is organized as follows. In Section II, maximum likelihood decoding is characterized as

Manuscript received January 19, 2012; revised August 8 and October 30, 2012; January 11, 2013. The associate editor coordinating the review of this paper and approving it for publication was A. Graell i Amat.

M. Esmaeili is with the Department of Mathematical Sciences, Isfahan University of Technology, 84156-83111, Isfahan, Iran, and the Dept. of Electrical and Computer Engineering, University of Victoria, Victoria, B.C., Canada (e-mail: emorteza@cc.iut.ac.ir, mesmaeil@ece.uvic.ca). He is also with the School of Mathematics, Institute for Research in Fundamental Sciences (IPM), P.O. Box: 19395-5746, Tehran, Iran.

A. Alampour is with the Department of Physics, Islamic Azad University, Shahreza Branch, Shahreza, Iran (e-mail: alialam154@yahoo.com).

T. A. Gulliver is with the Department of Electrical and Computer Engineering, University of Victoria, Victoria, BC, Canada (e-mail: agul-live@ece.uvic.ca).

This research was supported in part by a grant from IPM (No. 90050050). Digital Object Identifier 10.1109/TCOMM.2013.041113.120057

an integer programming problem, and the penalty function method is employed to obtain a simpler structure for the MLD problem. In Section III, the solution space of the penalized MLD problem is reduced to a ball and the main problem (MP) is formulated; this is referred to as problem MP. The properties of this problem are analyzed in Section IV. In Section V, the general local search algorithm for problem MP is described, and then this algorithm is simplified. Considering the decoder properties in Section V, a multi-flip local search algorithm is designed for problem MP in Section VI. These results are combined to obtain a local search decoder for the MLD problem in Section VII. In Section VIII, simulation results are given to illustrate the performance of the local search algorithm. These results are compared with those obtained using iterative probabilistic BP decoding. Finally, some conclusions are presented in Section IX.

II. INTEGER PROGRAMMING AND THE MLD PROBLEM

In this section, we present the general integer programming decoding problem over a binary symmetric channel (BSC) with crossover probability $\epsilon < 0.5$. Let $\mathbb{F}_2 := \{0, 1\}$, and consider an $[n, k, d]$ binary linear code $\mathcal{C} \subset \mathbb{F}_2^n$ with length n , dimension k and minimum Hamming distance d . Denote the parity check matrix of $\mathcal{C}$ by $H \in \mathbb{F}_2^{m \times n}$, where $m \geq n - k$. A binary column vector $\mathbf{x} \in \mathbb{F}_2^n$ is called a codeword if and only if $H\mathbf{x} = \mathbf{0}$. For a received word $\mathbf{y} \in \mathbb{F}_2^n$, the fixed vector $\lambda \in \{-1, 1\}^n$, with components $\lambda_i = (-1)^{y_i}$, $1 \leq i \leq n$, is called the scaled log likelihood ratio vector. With these assumptions, it can be shown that MLD is equivalent to finding the global solution of the following binary integer programming problem (and the solution is called the ML codeword)

$$\begin{aligned} & \text{minimize } \lambda^t \mathbf{x} \\ & \text{s.t. } H\mathbf{x} = \mathbf{0} \pmod{2}, \quad \mathbf{x} \in \mathbb{F}_2^n \end{aligned} \quad (1)$$

The MLD problem (1) belongs to the class of nonlinear integer programming (IP) problems. It was proven in [1] that the MLD problem is NP-hard. Therefore, heuristic algorithms are important tools to solve this problem. One such algorithm is the penalty function method, which is employed here to transfer constraints of problem (1) to the objective function. Denote the degree of check-node j by δ_j , $j = 1, \dots, m$. With this notation, for any parity check matrix H , assuming that the nonzero entries of the j th row of H are in positions $j_1, j_2, \dots, j_{\delta_j}$, we define

$$H_j(\mathbf{x}) = (x_{j_1} + x_{j_2} + \dots + x_{j_{\delta_j}}) \pmod{2}.$$

Then applying the penalty function method, problem (1) is replaced with the following equivalent problem

$$\begin{aligned} & \text{minimize } f(\mathbf{x}) = \lambda^t \mathbf{x} + p \sum_{j=1}^m H_j(\mathbf{x}) \\ & \text{s.t. } \mathbf{x} \in \mathbb{F}_2^n \end{aligned} \quad (2)$$

where p is a sufficiently large positive number referred to as the *penalty constant*. Hence, for sufficiently large p , problems (1) and (2) have the same optimal solution.

In problem (2), there are at least 2^k known local minimizers (i.e., the codewords), and for any codeword the second term is 0. For a received word $\mathbf{y}$ obtained via a BSC, the optimal solution of problem (2) is any codeword $\mathbf{c}$ with minimum

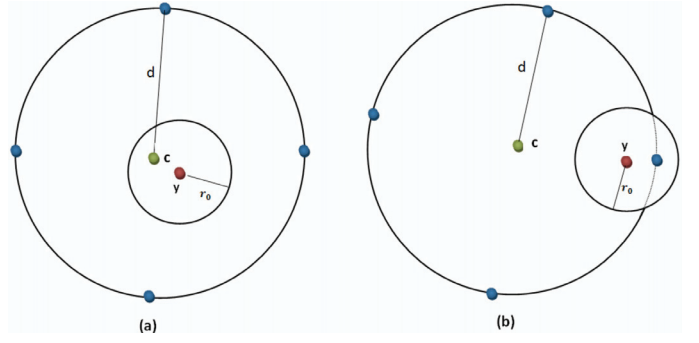


Fig. 1. Different locations of a received word $\mathbf{y}$ corresponding to a transmitted codeword $\mathbf{c}$.

distance from $\mathbf{y}$. The global solution of (2) is one of the 2^k codewords, and as k increases it becomes more difficult to find the global solution amongst the exponentially increasing number of local minimizers. To overcome this difficulty, consider that perturbations of a transmitted codeword $\mathbf{c}$ due to noise typically generate received words $\mathbf{y}$ close to $\mathbf{c}$. Thus, $\mathbf{c}$ should be close to $\mathbf{y}$ and by limiting the solution space to a fixed neighborhood of $\mathbf{y}$, we can reduce the number of codewords to be considered to those in this new space. In the next section, this approach is investigated in detail.

III. THE MLD PROBLEM ON A BALL

Consider an $[n, k, d]$ binary linear code $\mathcal{C}$, and let $r_0 = \lfloor \frac{d-1}{2} \rfloor$. Suppose that $\mathbf{y}$ is the received word corresponding to a transmitted codeword $\mathbf{c}$. For any $\mathbf{x}, \mathbf{x}' \in \mathbb{F}_2^n$, denote the Hamming distance between $\mathbf{x}$ and $\mathbf{x}'$ by $d(\mathbf{x}, \mathbf{x}')$. The closed ball $N_r(\mathbf{y})$ centered on $\mathbf{y}$ with radius r is defined by: $N_r(\mathbf{y}) = \{\mathbf{x} \in \mathbb{F}_2^n : d(\mathbf{x}, \mathbf{y}) \leq r\}$. Obviously, if $\mathbf{c} \in N_{r_0}(\mathbf{y})$ then it is the unique ML codeword. In other words, if at most r_0 errors occur, the only codeword that lies in $N_{r_0}(\mathbf{y})$ is the transmitted codeword $\mathbf{c}$. Define the error weight of codeword $\mathbf{c}$ as $w = d(\mathbf{y}, \mathbf{c})$. Figure 1 depicts a sphere of radius r_0 , and for a given radius r we have the following properties:

- 1) If $w \leq r_0$, there is only one codeword in $N_{r_0}(\mathbf{y})$, namely the transmitted codeword $\mathbf{c}$. This is illustrated in Fig. 1(a).
- 2) If $w > r_0$, then $\mathbf{c}$ does not belong to $N_{r_0}(\mathbf{y})$ and when searching $N_{r_0}(\mathbf{y})$ we will not find $\mathbf{c}$. Note that $N_{r_0}(\mathbf{y})$ contains at most one codeword. This is illustrated in Fig. 1(b).
- 3) If $r > r_0$ and $w < r$, then there is at least one codeword in $N_r(\mathbf{y})$, and when searching $N_r(\mathbf{y})$ the codeword obtained is not guaranteed to be $\mathbf{c}$.

Now, consider a BSC with crossover probability ϵ and a uniform input distribution. Let w_ϵ denote the highest recorded error weight in the transmission of N_w codewords over this channel. If $N_{w_\epsilon}(\mathbf{y})$ is assumed to be the decoding space of problem (2), then the probability that a transmitted codeword is not in $N_{w_\epsilon}(\mathbf{y})$ is less than $\frac{1}{N_w}$ with high probability if N_w is large enough, and as $N_w \rightarrow \infty$ this probability vanishes. In practice, for N_w sufficiently large the probability of an error weight greater than w_ϵ is negligible.

The solution space of problem (2) can therefore be reduced to a neighborhood of the received word $\mathbf{y}$ with radius w_ϵ

denoted $N_{w_\epsilon}(\mathbf{y})$. The first advantage of this approach is that the solution space of the problem is much smaller, compared to searching in the whole space $\mathbb{F}_2^n$, as finding a codeword in $N_{w_\epsilon}(\mathbf{y})$ with $\sum_{i=0}^{w_\epsilon} \binom{n}{i}$ elements is much easier than searching the space $\mathbb{F}_2^n$ with 2^n elements. However, the number of elements in $N_{w_\epsilon}(\mathbf{y})$ depends on n and w_ϵ . The second and main advantage of this approach is that the number of codewords in the reduced space is a small number as opposed to the 2^k codewords in $\mathbb{F}_2^n$. When w_ϵ is less than or equal to r_0 , there is at most one codeword in $N_{w_\epsilon}(\mathbf{y})$, and when w_ϵ is not much larger than r_0 , only a small number of codewords are in $N_{w_\epsilon}(\mathbf{y})$. Thus finding the global solution of problem (2) on $N_{w_\epsilon}(\mathbf{y})$ is much simpler than searching in $\mathbb{F}_2^n$.

A codeword belonging to $N_{r_0}(\mathbf{y})$ must be the ML codeword. As the error weight increases, the probability of having $\mathbf{c} \notin N_{r_0}(\mathbf{y})$ increases where $\mathbf{c}$ is the transmitted codeword, and $N_{r_0}(\mathbf{y})$ may not contain a codeword. In this case, neighborhoods of $\mathbf{y}$ with a larger radius can be searched. Note that when w_ϵ is greater than r_0 , there is no criterion that can guarantee $N_{w_\epsilon}(\mathbf{y})$ contains a unique codeword. Thus there may exist another codeword in $N_{w_\epsilon}(\mathbf{y})$ that is closer to the received word than the transmitted codeword, in which case the decoder will choose an incorrect codeword.

Considering the crossover probability of the channel, and setting $\mathbb{S} := N_{w_\epsilon}(\mathbf{y})$, we obtain the following main problem (MP) instead of problem (2)

$$\begin{aligned} & \text{minimize} \quad f(\mathbf{x}) = \lambda^t \mathbf{x} + p \sum_{j=1}^m H_j(\mathbf{x}) \\ & \text{s.t.} \quad \mathbf{x} \in \mathbb{S} \end{aligned} \quad (3)$$

In the rest of the paper, the term ‘problem MP’ refers to the problem defined by (3). In the next section, we investigate this problem.

IV. PROPERTIES OF PROBLEM MP

Definition 1: For any $\mathbf{u}, \mathbf{v} \in \mathbb{F}_2^n$, the binary sum of $\mathbf{u}$ and $\mathbf{v}$ is denoted by $\mathbf{u} \oplus \mathbf{v}$, the Hamming weight of $\mathbf{u}$ is defined by $w(\mathbf{u})$, and for any $\mathbf{x} \in \mathbb{F}_2^n$ the set $N_1(\mathbf{x}) = \{\mathbf{x}\} \cup \{\mathbf{x} \oplus \mathbf{e}_i : 1 \leq i \leq n, w(\mathbf{e}_i) = 1\}$ is called the neighborhood of $\mathbf{x}$.

Definition 2: A point $\mathbf{x}_0 \in \mathbb{S}$ is called a local minimizer of problem MP if $f(\mathbf{x}) \geq f(\mathbf{x}_0)$ for all $\mathbf{x} \in N_1(\mathbf{x}_0) \cap \mathbb{S}$. Similarly, $\mathbf{x}_0 \in \mathbb{S}$ is called a global minimizer of MP if $f(\mathbf{x}) \geq f(\mathbf{x}_0)$ for all $\mathbf{x} \in \mathbb{S}$.

Based on this definition, any global minimizer of problem MP is also a local minimizer.

Definition 3: For $1 \leq i \leq n$ and $\mathbf{x} \in \mathbb{S}$, a unit vector $\mathbf{e}_i \in \mathbb{F}_2^n$ is called a descent direction of function f at $\mathbf{x}$ if $\mathbf{x} \oplus \mathbf{e}_i \in \mathbb{S}$ and $f(\mathbf{x} \oplus \mathbf{e}_i) < f(\mathbf{x})$. Moreover, a descent direction $\mathbf{d}^*$ is called the steepest descent direction of f at $\mathbf{x}$ if for any descent direction $\mathbf{e}_i$ of f at $\mathbf{x}$ we have $f(\mathbf{x} \oplus \mathbf{d}^*) \leq f(\mathbf{x} \oplus \mathbf{e}_i)$. Note that $\mathbf{d}^*$ may not be unique, and any such $\mathbf{d}^*$ is referred to as a steepest descent direction of function f at $\mathbf{x}$.

If $\mathbf{x} \in \mathbb{F}_2^n$, it is not difficult to show that $|x_i - y_i| = y_i + \lambda_i x_i$, $1 \leq i \leq n$. Thus, $\sum_{i=1}^n |x_i - y_i| = \sum_{i=1}^n (y_i + \lambda_i x_i) = \sum_{i=1}^n y_i + \sum_{i=1}^n \lambda_i x_i$ and hence

$$d(\mathbf{x}, \mathbf{y}) = d(\mathbf{y}, \mathbf{0}) + \sum_{i=1}^n \lambda_i x_i = d(\mathbf{y}, \mathbf{0}) + \lambda^t \mathbf{x}. \quad (4)$$

Theorem 1: Let $\mathcal{C}$ be an $[n, k, d]$ binary code represented by a parity check matrix H with m rows and n nonzero columns, and minimum distance $d > 1$. Consider the associated problem MP with penalty constant p . Then

1. The integer $n + w_\epsilon + 1$ is a sufficient lower bound for the penalty constant p .
2. $\mathbf{x} \in \mathbb{S}$ is a codeword if and only if $f(\mathbf{x}) \leq w_\epsilon$.
3. If $\mathbf{x} \in \mathbb{S}$ is a codeword, then there is no codeword in $N_1(\mathbf{x}) \cap \mathbb{S}$.
4. Every codeword $\mathbf{x} \in \mathbb{S}$ is a local minimizer of problem MP.

Proof:

1. For all $1 \leq i \leq m$, we know $H_i(\mathbf{x}) \in \{0, 1\}$, and $f(\mathbf{x}) = \lambda^t \mathbf{x} + p \sum_{i=1}^m H_i(\mathbf{x})$. Moreover, for any $\mathbf{x} \in \mathbb{S} = N_{w_\epsilon}(\mathbf{y})$ we have $0 \leq d(\mathbf{x}, \mathbf{y}) \leq w_\epsilon$. By considering (4), we have $0 \leq d(\mathbf{y}, \mathbf{0}) + \lambda^t \mathbf{x} \leq w_\epsilon$ and then $-d(\mathbf{y}, \mathbf{0}) \leq \lambda^t \mathbf{x} \leq w_\epsilon - d(\mathbf{y}, \mathbf{0})$. Thus

$$-n \leq \lambda^t \mathbf{x} \leq w_\epsilon. \quad (5)$$

It must be that p is a positive number. If $\mathbf{x}$ is not a codeword then $p \sum_{i=1}^m H_i(\mathbf{x}) \geq p$ and hence assuming that p is large enough, (5) implies that $f(\mathbf{x}) > w_\epsilon$ (note that (5) implies that $\lambda^t \mathbf{x}$ is bounded below by $-n$). Moreover, there exists an integer $1 \leq k \leq m$ such that $f(\mathbf{x}) = \lambda^t \mathbf{x} + kp$. Thus, $\lambda^t \mathbf{x} + kp > w_\epsilon$ and we have $p > \frac{w_\epsilon - \lambda^t \mathbf{x}}{k}$. Besides, $w_\epsilon - \lambda^t \mathbf{x} \geq 0$ and $\frac{w_\epsilon - \lambda^t \mathbf{x}}{k} < w_\epsilon - \lambda^t \mathbf{x} \leq w_\epsilon + n$. Therefore, if $p \geq w_\epsilon + n + 1$ then $p > \frac{w_\epsilon - \lambda^t \mathbf{x}}{k}$. This shows that $w_\epsilon + n + 1$ is a sufficient lower bound for the penalty constant p .

2. If $\mathbf{x} \in \mathbb{S}$ is a codeword, then $\sum_{i=1}^m H_i(\mathbf{x}) = 0$. Hence considering (5), it is obvious that $f(\mathbf{x}) \leq w_\epsilon$. Conversely, if $f(\mathbf{x}) \leq w_\epsilon$ and $\mathbf{x}$ is not a codeword, then at least one of the parity check equations is not satisfied, and $f(\mathbf{x}) = \lambda^t \mathbf{x} + p \leq w_\epsilon$. According to the first statement $p > w_\epsilon + n$, so that $\lambda^t \mathbf{x} < -n$, which contradicts (5).

3. If $\mathbf{x}$ is a codeword, then $d > 1$ implies that no other element of $N_1(\mathbf{x})$ is a codeword.

4. Suppose that $\mathbf{x} \in \mathbb{S}$ is a codeword. If $\mathbf{x}' \in N_1(\mathbf{x})$, then there exists a j such that $x' = 1 - x_j$, and therefore

$$\begin{aligned} f(\mathbf{x}) &= \lambda^t \mathbf{x} = \lambda^t \mathbf{x}' + \lambda_j x_j - \lambda_j x'_j = \lambda^t \mathbf{x}' + \lambda_j (2x_j - 1) \\ &< \lambda^t \mathbf{x}' + 1 < \lambda^t \mathbf{x}' + p < f(\mathbf{x}'). \end{aligned}$$

which implies that $\mathbf{x}$ is a local minimizer of problem MP. ■

Finding an optimal solution of problem MP requires a search technique. In the next section, a local search method is considered to find a local minimizer of this optimization problem.

V. LOCAL SEARCH FOR PROBLEM MP

To find a local minimizer of problem MP, we employ a search technique which is known in combinatorial optimization as *local search* [13]. The local search algorithm is described below.

Algorithm 1 (Local Search)

Choose an initial point $\mathbf{x} \in \mathbb{S}$.

while $\mathbf{x}$ is not a local minimizer of problem MP **do**

take a steepest descent direction $\mathbf{d}^*$ of f at $\mathbf{x}$.

If $\mathbf{x} \oplus \mathbf{d}^* \in \mathbb{S}$ then set $\mathbf{x} := \mathbf{x} \oplus \mathbf{d}^*$.

end while

$\mathbf{x}$ is local minimizer of problem MP.

STOP.

Now, consider problem MP for a given crossover probability ϵ , w_ϵ and $\mathbf{y}$. In Algorithm 1, we must determine whether or not $\mathbf{x} := \mathbf{x} \oplus \mathbf{d}^*$ is in $\mathbb{S}$. This requires the calculation of $d(\mathbf{x}, \mathbf{y})$, as $d(\mathbf{x}, \mathbf{y}) \leq w_\epsilon$ if and only if $\mathbf{x} \in \mathbb{S}$. Fortunately, previous results can be employed to accelerate this determination. To see this, suppose $\mathbf{x} \in \mathbb{S}$. Since $d(\mathbf{x}, \mathbf{y})$ is known and $d(\mathbf{x}, \mathbf{y}) \leq w_\epsilon$, for any $\mathbf{x}' \in N_1(\mathbf{x})$ there exists a j such that $x'_j = 1 - x_j$ and

$$\begin{aligned} d(\mathbf{x}', \mathbf{y}) &= \sum_{i=1}^n |x'_i - y_i| = \\ &= \sum_{i=1}^n |x_i - y_i| + |x'_j - y_j| - |x_j - y_j| = \\ &= d(\mathbf{x}, \mathbf{y}) + |1 - x_j - y_j| - |x_j - y_j|. \end{aligned}$$

For $\mathbf{x}, \mathbf{y} \in \mathbb{F}_2^n$, it is obvious that $|1 - x_j - y_j| = 1 - |x_j - y_j|$. Therefore, $d(\mathbf{x}', \mathbf{y})$ can be calculated simply as follows

$$d(\mathbf{x}', \mathbf{y}) = d(\mathbf{x}, \mathbf{y}) + 1 - 2|x_j - y_j|. \quad (6)$$

Consider again problem MP. As mentioned before, when using Algorithm 1 to find a local minimizer, $f(\mathbf{x})$ must be calculated many times. To overcome this issue, the calculation of $f(\mathbf{x})$ can be simplified using previous results, as shown below.

For variable node i , $1 \leq i \leq n$, let $\mathbf{V}_i$ be the vector giving the location of the nonzero elements of the i th column of H . In other words, $\mathbf{V}_i$ includes the numbers of the parity check equations that contain variable x_i . By considering (6), if $\mathbf{x}' \in N_1(\mathbf{x}) \cap \mathbb{S}$, then there is an index $1 \leq i \leq n$ such that $x'_i = 1 - x_i$. This implies that

$$\begin{aligned} f(\mathbf{x}') &= \lambda^t \mathbf{x}' + p \sum_{j=1}^m H_j(\mathbf{x}') = \\ &= \lambda^t \mathbf{x} + \lambda_i x'_i - \lambda_i x_i + p \sum_{j=1}^m H_j(\mathbf{x}'). \end{aligned}$$

By replacing x'_i with $1 - x_i$, after simplification we obtain

$$f(\mathbf{x}') = \lambda^t \mathbf{x} + \lambda_i(1 - 2x_i) + p \sum_{j=1}^m H_j(\mathbf{x}').$$

If x_i is flipped to $1 - x_i$, all parity check equations in $\mathbf{V}_i$ should also be flipped, and

$$f(\mathbf{x}') = \lambda^t \mathbf{x} + \lambda_i(1 - 2x_i) + p \sum_{j \notin \mathbf{V}_i} H_j(\mathbf{x}) + p \sum_{j \in \mathbf{V}_i} H_j(\mathbf{x}'),$$

which can be written as

$$\begin{aligned} f(\mathbf{x}') &= \lambda^t \mathbf{x} + \lambda_i(1 - 2x_i) + \\ &+ p \sum_{j=1}^m H_j(\mathbf{x}) + p \sum_{j \in \mathbf{V}_i} (H_j(\mathbf{x}') - H_j(\mathbf{x})). \end{aligned}$$

Considering the definition of $f(\mathbf{x}) = \lambda^t \mathbf{x} + p \sum_{j=1}^m H_j(\mathbf{x})$, we get

$$f(\mathbf{x}') = f(\mathbf{x}) + \lambda_i(1 - 2x_i) + p \sum_{j \in \mathbf{V}_i} (H_j(\mathbf{x}') - H_j(\mathbf{x})).$$

We know that for $j \in \mathbf{V}_i$

$$H_j(\mathbf{x}') = \cdots \oplus x'_i \oplus \cdots = \cdots \oplus 1 \oplus x_i \oplus \cdots = 1 \oplus H_j(\mathbf{x}).$$

In addition, $1 \oplus H_j(\mathbf{x}) = 1 - H_j(\mathbf{x})$, so that

$$f(\mathbf{x}') = f(\mathbf{x}) + \lambda_i(1 - 2x_i) + p \sum_{j \in \mathbf{V}_i} (1 - 2H_j(\mathbf{x})). \quad (7)$$

Therefore in (7), for any $\mathbf{x}' \in N_1(\mathbf{x})$ we can easily calculate $f(\mathbf{x}')$ from $f(\mathbf{x})$ by simply adding it to the term $\lambda_i(1 - 2x_i) + p \sum_{j \in \mathbf{V}_i} (1 - 2H_j(\mathbf{x}))$.

Considering the results in (6) and (7), Algorithm 1 for problem MP can be revised as follows.

Algorithm 2

Initialization: Input n, m and w_ϵ . Choose an initial point $\mathbf{x} \in \mathbb{S}$. For $1 \leq j \leq m$, set $h_j := H_j(\mathbf{x})$, $a := d(\mathbf{x}, \mathbf{y})$, $f_0 := \lambda^t \mathbf{x} + p \sum_{j=1}^m h_j$, $f_{\min} := f_0$, and $i := 1$.

do

repeat

if $a + 1 - 2|x_i - y_i| \leq w_\epsilon$ **then**

denote $f' := f_0 + \lambda_i(1 - 2x_i) + p \sum_{j \in \mathbf{V}_i} (1 - 2h_j)$;

if $f' < f_{\min}$ **then**

set $f_{\min} := f'$ and $l := i$;

end if

end if

set $i := i + 1$;

until $i \leq n$.

if $f_{\min} = f_0$ exit from **do** loop;

set $f_0 := f_{\min}$, $a := a + 1 - 2|x_l - y_l|$, $x_l := 1 - x_l$;

for all $j \in \mathbf{V}_l$ set $h_j := 1 - h_j$; $i := 1$;

end do

$\mathbf{x}$ is a local minimizer of problem MP.

STOP.

Algorithm 2 starts from an arbitrary point and flips a single bit until a local minimizer of problem MP is found. The main goal of this algorithm is to find a codeword in $\mathbb{S}$. Empirical results show that when w_ϵ is not far from r_0 , if this algorithm produces a codeword it is almost always the correct codeword.

Although this modified local search algorithm is faster than the original local search algorithm, in an iteration only a single bit is flipped corresponding to a steepest descent direction. In practice, the steepest descent direction may not be unique so there are multiple choices to flip. In the next section, we describe a new multi-flip local search algorithm which is faster than Algorithm 2.

VI. MULTI-FLIP LOCAL SEARCH FOR PROBLEM MP

Algorithm 2 starts from an initial point and sequentially searches neighborhoods of steepest decent directions until a local minimizer is obtained. In an iteration of Algorithm 2, the steepest descent direction need not be unique, in which case more than one bit can be flipped to accelerate the determination of a local minimizer.

Consider $\mathbf{x} \in \mathbb{S}$ as a current point of Algorithm 2. If $\mathbf{x}$ is not a local minimizer, then there is at least one steepest descent direction. Denote the number of steepest descent directions as l and the index set of these directions as $\mathcal{D} := \{d_1, d_2, \dots, d_l\}$. In other words, in a neighborhood of a point in an iteration of Algorithm 2, there may be more than one direction in which to

flip a bit. Compared with the single bit flipping in Algorithm 2, a multi-flip strategy can be used to provide a faster solution. Thus Algorithm 2 is modified as follows. If there is only one steepest descent direction in an iteration, Algorithm 3 is the same as Algorithm 2. Conversely, if the steepest descent direction is not unique, for all points indexed in $\mathcal{D}$ Algorithm 3 computes the related objective function using (7), and when the value of this function decreases the corresponding bit is flipped.

Algorithm 3

Initialization: Input n, m and w_ϵ . Choose an initial point $\mathbf{x} \in \mathbb{S}$. For $1 \leq j \leq m$, set $h_j := H_j(\mathbf{x})$, $a := d(\mathbf{x}, \mathbf{y})$, $f_0 := \lambda^t \mathbf{x} + p \sum_{j=1}^m h_j$, $f_{\min} := f_0$, $\mathcal{D} := \emptyset$, and $i := 1$.

```

do
  repeat
    if  $a + 1 - 2|x_i - y_i| \leq w_\epsilon$  then
      denote  $f' := f_0 + \lambda_i(1 - 2x_i) + p \sum_{j \in \mathbf{V}_i} (1 - 2h_j)$ ;
      if  $f' < f_{\min}$  then
        set  $\mathcal{D} := \emptyset$ , and  $f_{\min} := f'$ ;
      end if
      if  $f' = f_{\min}$  add  $i$  to  $\mathcal{D}$ ;
    end if
    set  $i := i + 1$ ;
  until  $i \leq n$ 
  if  $\mathcal{D} = \emptyset$  exit from do loop;
  set  $\mathcal{D} := \{d_1, d_2, \dots, d_l\}$ ,  $f_0 := f_{\min}$ ,
   $a := a + 1 - 2|x_{d_1} - y_{d_1}|$ , and  $x_{d_1} := 1 - x_{d_1}$ ;
  if  $f_0 \leq w_\epsilon$  exit from do loop;
  for all  $j \in \mathbf{V}_{d_1}$  update  $h_j := 1 - h_j$ ; and set  $k := 1$ ;
  while  $k \leq l$  do
    if  $a + 1 - 2|x_{d_k} - y_{d_k}| \leq w_\epsilon$  then
      denote  $f' := f_0 + \lambda_{d_k}(1 - 2x_{d_k}) + p \sum_{j \in \mathbf{V}_{d_k}} (1 - 2h_j)$ ;
      if  $f' \leq f_0$  then
        for all  $j \in \mathbf{V}_{d_k}$  update  $h_j := 1 - h_j$ ;
         $f_0 := f'$ ,  $f_{\min} := f_0$ ,  $a := a + 1 - 2|x_{d_k} - y_{d_k}|$ ,
        and  $x_{d_k} := 1 - x_{d_k}$ ;
        if  $f_0 \leq w_\epsilon$  exit from do loop;
      end if
    end if
    set  $k := k + 1$ ;
  end while
end do
 $\mathbf{x}$  is a local minimizer of problem MP.
STOP.

```

The main goal in employing local search is to find the transmitted codeword $\mathbf{c}$. When the crossover probability ϵ is small, the received word $\mathbf{y}$ is very likely close to this codeword, thus $\mathbf{y}$ is a good initial point for starting the local search. Points near $\mathbf{y}$ are also good choices as they have the potential to decrease the objective function. In the next section, we combine all of the concepts considered thus far to obtain a local search based decoder for binary linear codes over a BSC.

VII. LOCAL SEARCH DECODER FOR THE MLD PROBLEM

Consider an $[n, k, d]$ code $\mathcal{C}$ and a BSC with crossover probability ϵ . In addition, let $\mathbf{y}$ be the received word and w_ϵ the maximum estimated error weight. The corresponding problem

MP is now constructed. When ϵ is small, the transmitted codeword $\mathbf{c}$ is typically not far from $\mathbf{y}$, so $\mathbf{y}$ is a good choice for the initial point in Algorithm 3. If at the first stage Algorithm 3 produces a codeword, it is accepted as the transmitted codeword. However, if the algorithm fails to find a codeword, the initial point must be changed. A point should be chosen which has the potential to reduce the objective function of problem MP, and it should not be far from $\mathbf{y}$. Towards this goal, define the index set of all descent directions at $\mathbf{y}$ as $A = \{i | f(\mathbf{x}_i) < f(\mathbf{y}), \mathbf{x}_i \in N_1(\mathbf{y}), i = 1, 2, \dots, n\}$, and arrange the elements of A by steepness in ascending order such that for any two consecutive indices i and i' in A we have $f(\mathbf{x}_i) \leq f(\mathbf{x}_{i'})$. Denote the number of elements in A by $|A|$. It is obvious that among all points obtained from the directions indexed in A , $\mathbf{x}_1$ should be considered first and $\mathbf{x}_{|A|}$ considered last in attempting to decrease $f(\mathbf{x})$. Since the steepest descent directions may not be unique, there may be more than one point with highest priority. Thus, we define the index set of all directions with highest priority (greatest potential to decrease $f(\mathbf{x})$ at $\mathbf{y}$), as $B = \{i \in A | f(\mathbf{x}_i) = f(\mathbf{x}_1)\}$. For the second stage of Algorithm 3, the following three types of initial points are defined:

- 1) $\mathbf{x}$ obtained from $\mathbf{y}$ after flipping all indices in B and one index in $A - B$;
- 2) $\mathbf{x}$ obtained from $\mathbf{y}$ after flipping $|B| - 1$ indices in B and one index in $A - B$;
- 3) $\mathbf{x}$ obtained from $\mathbf{y}$ after flipping one index in A .

These initial points are not far from $\mathbf{y}$ and have the potential to decrease $f(\mathbf{x})$, thus they are all good initial points for Algorithm 3. If the second stage of Algorithm 3 (using the above initial points) fails to find a codeword, we chose at most N initial points randomly from $\mathbb{S} - N_1(\mathbf{y})$. If after these N iterations a codeword is not found, a decoding failure is declared. A random initial point $\mathbf{x}$ in $\mathbb{S} - N_1(\mathbf{y})$ is chosen by selecting a random integer $2 \leq r \leq w_\epsilon$, and flipping r bits of $\mathbf{y}$ using a uniform distribution.

Algorithm 4 (Multi-Flip Local Search Decoding)

```

Initialization: Input  $\mathbf{y}, n, m, N$  and  $w_\epsilon$ . Set  $p := n + w_\epsilon + 1$ ,
 $\mathbb{S} := N_{w_\epsilon}(\mathbf{y})$ , and construct the corresponding problem MP.
if syndrome( $\mathbf{y}$ )  $\neq \mathbf{0}$  then
  execute Algorithm 3 using the initial point  $\mathbf{y}$ ;
  if the result is not a codeword then
    construct the sets  $A$  and  $B$ ;
    repeat for the three types of initial points and  $N$  random
    initial points
      execute Algorithm 3 starting from an initial point;
    until the decoding result is a codeword or the limit on
    the number of initial points is reached;
    if the result is not a codeword then
      the algorithm did not find a codeword;
      STOP.
    end if
  end if
end if
Decoding was successful and a codeword was found.
STOP.

```

The complexity of Algorithm 4 depends on the code length n , the maximum error weight w_ϵ , and the number of initial

points, in particular N . When w_ϵ is not far from r_0 , Algorithm 3 will almost always find the original codeword starting from $\mathbf{y}$ or an initial point in $N_1(\mathbf{y}) - \{\mathbf{y}\}$ in the second stage, and typically only a few random points in $\mathbb{S} - N_1(\mathbf{y})$ are needed in the third stage. When w_ϵ is far from r_0 , more random points are needed in the third stage, which increases the computational complexity and reduces the accuracy of Algorithm 4.

Some examples are presented in the next section to illustrate the performance of this algorithm. Further, this local search decoder (LSD) is compared with a belief propagation decoder (BPD) in terms of word error rate (WER) and decoding time.

VIII. NUMERICAL RESULTS

In this section, we investigate the performance of the proposed LSD given by Algorithm 4 for a BSC with different crossover probabilities. The performance with the LSD is compared with that of a probabilistic BPD. The probabilistic decoding algorithm presented in [15] and implemented in [16] was used for the BPD simulation with a maximum of 100 iterations. The simulations were done for both decoders with programs written in C++ using the Microsoft Visual Studio 2010 compiler.

As mentioned in Section 3, to estimate w_ϵ for a given crossover probability ϵ , the all-zero codeword is transmitted at least $N_\epsilon = 10^6$ times over the BSC. In the figures, WER denotes the word error rate or the average number of decoding errors using the decoders, and $T(s)$ is the average decoding time per codeword in seconds.

Example 1: Consider the $[155, 64, 20]$ Tanner code and set $N = 15000$.

TABLE I
CROSSOVER PROBABILITIES AND THE CORRESPONDING MAXIMUM ERROR WEIGHTS FOR EXAMPLE 1.

ϵ	0.02	0.025	0.03	0.035	0.04	0.045	0.05	0.055
w_ϵ	16	17	18	20	21	22	24	25

Example 2: Consider the $[504, 252]$ MacKay code given in [14], and set $N = 15000$. The minimum distance of this code is 20 [18].

TABLE II
CROSSOVER PROBABILITIES AND THE CORRESPONDING MAXIMUM ERROR WEIGHTS FOR EXAMPLE 2.

ϵ	0.011	0.012	0.014	0.016	0.018	0.02
w_ϵ	22	24	25	27	29	32

Example 3: Consider the M-SC-MPC $[840, 702, 10]$ code given in [17], and set $N = 10000$.

TABLE III
CROSSOVER PROBABILITIES AND THE CORRESPONDING MAXIMUM ERROR WEIGHTS FOR EXAMPLE 3.

ϵ	0.0005	0.001	0.0015	0.002	0.0025	0.003
w_ϵ	7	8	10	12	14	16

TABLE IV
CROSSOVER PROBABILITIES AND THE CORRESPONDING MAXIMUM ERROR WEIGHTS FOR EXAMPLE 4.

ϵ	0.001	0.0015	0.002	0.0025	0.003
w_ϵ	13	18	20	22	24

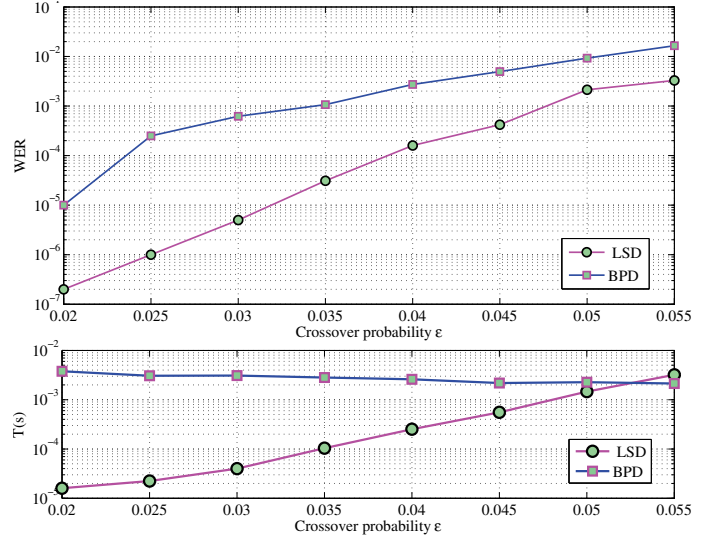


Fig. 2. WERs and decoding times for Example 1.

Example 4: Consider the regular $(4,36)$ MacKay code (s2.94.594.gz) with $n = 1998$ and rate $R = 0.8889$ given in [14], and set $N = 20000$.

The results given in Examples 1–4 show that our LSD has better performance than the BPD in the error floor region. Moreover, in this region the LSD decoding time is considerably less than that with the BPD. In Example 1, the minimum distance of the code is 20, $r_0 = 9$, and for $w_\epsilon = 16$ with $\epsilon = 0.02$ and $w_\epsilon = 17$ with $\epsilon = 0.025$, the WER is below 10^{-6} . Considering the first three examples, we conjecture that in the region with radius slightly higher than r_0 , the performance of the proposed LSD algorithm is similar to that with an ML decoder.

IX. CONCLUSION

In this paper, the problem of hard decision maximum-likelihood decoding of binary linear block codes was formulated as an integer program using the penalty function method. Instead of solving this optimization problem on $\mathbb{F}_2^n$, the solution space was restricted to a small ball, and a main problem MP formulated to solve the resulting maximum likelihood problem. Using the properties of this problem, a modified general local search algorithm was presented. To lower the computational complexity, a multi-flip algorithm was given to find a local minimizer of problem MP. Finally, the properties of problem MP and the multi-flip algorithm were exploited to design a multi-flip local search decoder (LSD).

Performance results were presented which show that when the radius of the solution space $\mathbb{S}$ is near the minimum distance of the code, the performance of the LSD is better than that of the BPD. The decoding time of the LSD is less than that of the BPD in the error floor region. In particular, compared

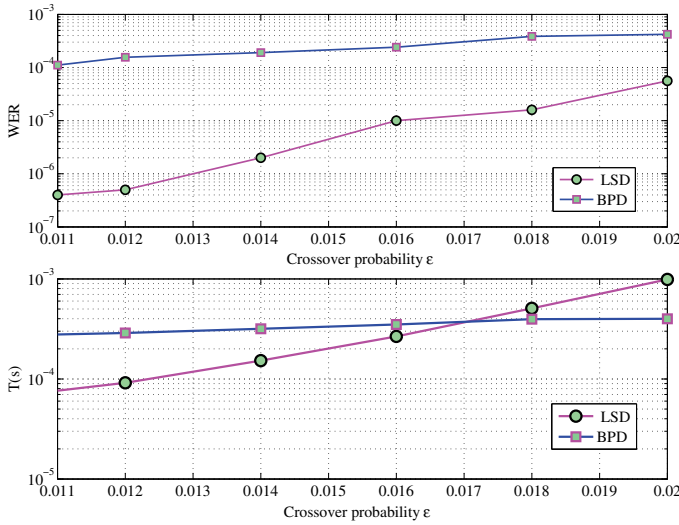


Fig. 3. WERs and decoding times for Example 2.

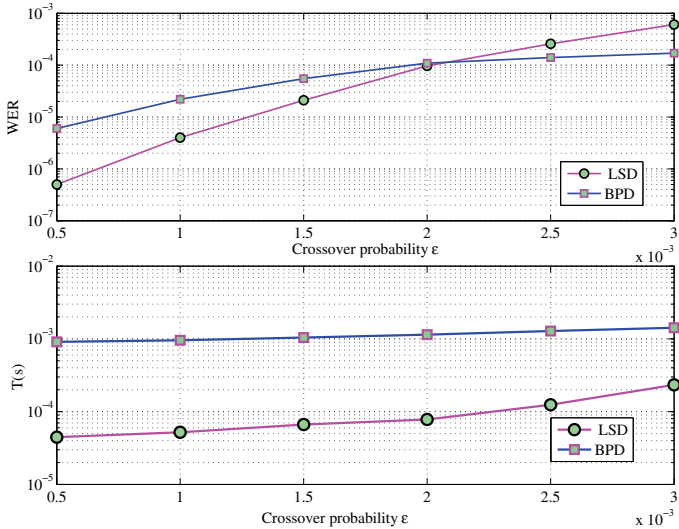


Fig. 4. WERs and decoding times for Example 3.

to BPD, when the radius of the solution space $\mathbb{S}$ is close to $r_0 = \lfloor \frac{d-1}{2} \rfloor$, the performance of the LSD is very good and the corresponding decoding time is considerably less than that for the BPD.

ACKNOWLEDGMENT

The authors would like to thank the anonymous referees for their constructive comments that substantially improved the quality and presentation of this paper.

REFERENCES

- [1] E. Berlekamp, R. McEliece, and H. van Tilborg, "On the inherent intractability of certain coding problems," *IEEE Trans. Inf. Theory*, vol. 24, no. 3, pp. 954–972, May 1978.
- [2] M. Breibach, M. Bossert, R. Lucas, and C. Kemper, "Soft-decision decoding of linear block codes as optimization problem," *Eur. Trans. Telecommun.*, vol. 9, no. 3, pp. 289–293, May–June 1998.
- [3] J. Feldman, M. J. Wainwright, and D. R. Karger, "Using linear programming to decode binary linear codes," *IEEE Trans. Inf. Theory*, vol. 51, no. 3, pp. 954–972, May 2005.

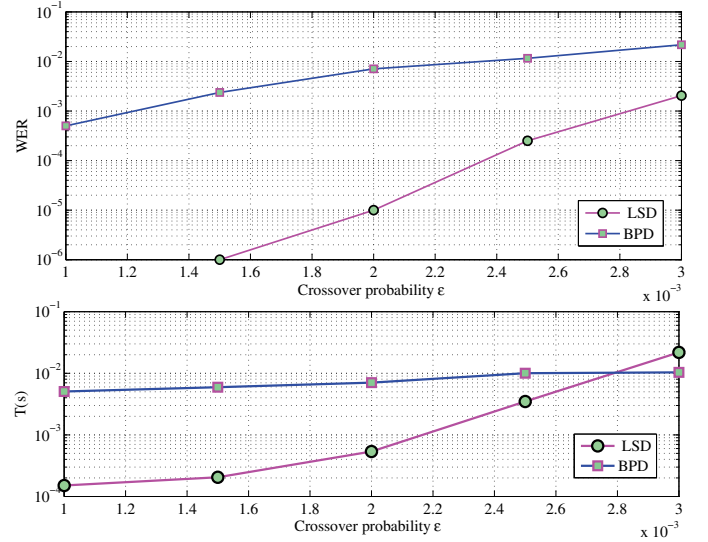


Fig. 5. WERs and decoding times for Example 4.

- [4] K. Yang, X. Wang, and J. Feldman, "A new linear programming approach to decoding linear block codes," *IEEE Trans. Inf. Theory*, vol. 54, no. 3, pp. 1064–1072, Mar. 2008.
- [5] A. Tanatmis, S. Ruzika, H. W. Hamacher, M. Punekar, F. Kienle, and N. Wehn, "A separation algorithm for improved LP-decoding of linear block codes," *IEEE Trans. Inf. Theory*, vol. 56, no. 7, pp. 3277–3289, July 2010.
- [6] A. Tanatmis, S. Ruzika, M. Punekar, and F. Kienle, "Numerical comparison of IP formulations as ML decoders," in *Proc. 2010 IEEE Int. Conf. Commun.*, pp. 1–5.
- [7] M. Helmling, S. Ruzika, and A. Tanatmis, "Mathematical programming decoding of binary linear codes," *IEEE Trans. Inf. Theory*, vol. 58, no. 7, pp. 4753–4769, July 2012.
- [8] F. R. Kschischang, B. J. Frey, and H. A. Loeliger, "Factor graphs and the sum-product algorithm," *IEEE Trans. Inf. Theory*, vol. 47, no. 2, pp. 498–519, Feb. 2001.
- [9] N. Miladinovic and M. P. C. Fossorier, "Improved bit-flipping decoding of low-density parity-check codes," *IEEE Trans. Inf. Theory*, vol. 51, no. 4, pp. 1594–1606, Apr. 2005.
- [10] M. Jiang, C. Zhao, Z. Shi, and Y. Chen, "An improvement on the modified weighted bit flipping decoding algorithm for LDPC codes," *IEEE Commun. Lett.*, vol. 9, no. 9, pp. 814–816, Sept. 2005.
- [11] T.-C. Chen, "An efficient bit-flipping decoding algorithm for LDPC codes," in *Proc. 2012 Cross Strait Quad-Regional Radio Science and Wireless Tech. Conf.*, pp. 109–112.
- [12] H. Kamabe and S. Kobota, "Simple improvements of bit-flipping decoding," in *Proc. 2010 Int. Conf. on Advanced Commun. Tech.*, pp. 113–118.
- [13] M. Pirlot, "General local search methods," *Eur. J. Oper. Res.*, vol. 92, no. 3, pp. 493–511, Aug. 1996.
- [14] D. J. C. MacKay, *Encyclopedia of Sparse Graph Codes*. Available: <http://www.inference.phy.cam.ac.uk/mackay/codes/data.html/>
- [15] R. H. Morelos-Zaragoza, *The Art of Error Correcting Coding*. Available: <http://the-art-of-ecc.com/>
- [16] R. H. Morelos-Zaragoza, *The Art of Error Correcting Coding*. Wiley, 2006.
- [17] M. Baldi, G. Cancellieri, and F. Chiaraluce, "Good LDPC codes based on very simple component codes," in *Proc. 2009 Assoc. of Telecommun. and Inform. Tech. Group (GTIT) Annual Meeting*, pp. 1–8.
- [18] E. Rosnes and Ø. Ytrehus, "An efficient algorithm to find all small-size stopping sets of low-density parity-check matrices," *IEEE Trans. Inf. Theory*, vol. 55, no. 9, pp. 4167–4178, Sept. 2009.



Morteza Esmaili (M09) received the M.Sc. degree in mathematics in 1988 from Kharazmi University (formerly, the Teacher Training University of Tehran), Tehran, Iran, and the Ph.D. degree in mathematics (coding theory) in 1996 from Carleton University, Ottawa, ON, Canada. The following two years he was a postdoctoral fellow in the Department of Electrical and Computer Engineering at the University of Waterloo, Waterloo, ON, Canada. He joined Isfahan University of Technology in September 1998, where he is now a Professor in the

Department of Mathematical Sciences. Dr. Esmaili has been an Adjunct Professor in the Department of Electrical and Computer Engineering at the University of Victoria, Victoria, BC, Canada, since July 2009. His current research interests include coding and information theory, cryptography, and combinatorics.



A. Alampour received the M.Sc. degree in applied mathematics (Operational Research and Optimization) from Shahid Chamran University of Ahvaz, Ahvaz, Iran. His research interests focus on optimal shape design, optimization, iterative decoding and coding theory.



T. Aaron Gulliver (SM96) received the Ph.D. degree in Electrical Engineering from the University of Victoria, Victoria, BC, Canada in 1989. From 1989 to 1991 he was employed as a Defence Scientist at Defence Research Establishment Ottawa, Ottawa, ON, Canada. He has held academic positions at Carleton University, Ottawa, and the University of Canterbury, Christchurch, New Zealand. He joined the University of Victoria in 1999 and is a Professor in the Department of Electrical and Computer Engineering. In 2002, he became a Fellow of the Engineering Institute of Canada, and in 2012 a Fellow of the Canadian Academy of Engineering. He is currently an Area Editor for IEEE TRANSACTIONS ON WIRELESS COMMUNICATIONS. From 2000–2003, he was Secretary and a member of the Board of Governors of the IEEE Information Theory Society. His research interests include information theory and communication theory, algebraic coding theory, smart grid and ultrawideband communications.